1 - React Related JavaScript Refresher

Arrow function and Function Expressions

These are a more concise way to write function and are used to define function component.

Regular Function Declaration:

```
function getRectangleArea(width, height){
    return width * height;
}
```

Same in the Arrow function:

```
const getRectangleArea = (width, height) => {
    return width * height;
};
```

since the function body is a single expression, you can omit the curly braces and the return keyword.

```
const getRectangleArea = (width, height) => width * height;
```

you can also use Arrow functions as Callbacks

```
const numbers = [1, 2, 3, 4, 5];
const doubled = numbers.map((number) => number * 2);
```

The Difference between Arrow function and Regular function.

1) hoisting

```
regular();
arrow();

function regular(){
    console.log('Regular');
}

const arrow = () => console.log('Arrow');
```

↳ while running one will see "Regular" printed fine but you get an error for the arrow function, because we are trying to access it before it was initialized.

this keyword Difference

with normal function, the this variable is created which references the objects that call them. Arrow function do create their own this binding

```
const person = {
    name: 'Brad',
    sayHelloRegular: function(){
        console.log("Regular:", this.name);
    };
    sayHelloArrow: ()=>{
        console.log('Arrow:', this.name),
    };
};
```

The regular function points to the person objects. The arrow function will give you undefined. ↘ the reason.

Run the code and you get the person object for the regular function and either an empty object (Node.js) or the `window` object (Browser). This is because the arrow function does not automatically create a `this` variable. As a result, any reference to this would point to what this was before the function was created. In Node with ES Modules, it's an empty object, in the browser, it's the `window` object.

Template Literals

↳ Basically this

```
const name = 'John';
// Concatenation using the + operator
const greeting = 'hello ,' + name

// using Template Literals
const greeting = `Hello , ${name}`;
```

Ternery Operator & Short Circuit Rendering

1) Ternary Operator

↳ is a concise way to write an if statement in JavaScript

here an Example

```
const number = 5;
let message;
if (number % 2 === 0) {
    message = 'Even Number';
} else {
   message = 'Odd Number';
}
```

```
Console.log(message);
```

Let rewrite this using the ternary Operator

```
const number = 5;
const message = number % 2 === 0 ? 'Even Number' : 'Odd message';
console.log(message);
```

2) Short Circuit Rendering

↳ is a another way to conditionally render in React. It's based of short circuit evaluation in JavaScript. In short circuit evaluation the evaluated is only evaluated if the first Operand in not falsy. In React we use as sort of ternery operator without the else part.